# FreeLedger

## Technical Documentation

Blockchain-Based Freelance Escrow Platform

---

**Author:** Pawan Poudel

July 16, 2026

### Document Information

| | |
|---|---|
| **Project** | FreeLedger — Gig Economy Escrow Platform |
| **Version** | 3.0 (Capstone Final) |
| **Stack** | React, FastAPI, PostgreSQL, Solidity, IPFS |
| **Blockchain** | Ethereum (Hardhat Local Network) |
| **License** | MIT |

This document provides a comprehensive technical overview of the FreeLedger platform, covering system architecture, smart contract design, API reference, and core business flows.

# Contents

# 1 Introduction

## 1.1 Project Overview

FreeLedger is a decentralized freelance marketplace that uses Ethereum smart contracts to enforce escrow-based payments between clients and freelancers. The platform eliminates the need for trusted intermediaries by holding client funds in a smart contract that automatically releases payment upon milestone approval.

## 1.2 Core Value Proposition

- **Trustless Escrow:** Client funds are locked in a smart contract, not held by the platform.
- **Milestone-Based Payments:** Granular payment release tied to deliverable verification.
- **Immutable Audit Trail:** All contract state changes are recorded on-chain.
- **Dispute Resolution:** Platform administrators can adjudicate disputes with on-chain fund release.
- **IPFS Integration:** Contract terms and deliverables stored on the InterPlanetary File System.

## 1.3 Technology Stack

| Layer | Technology | Version |
|---|---|---|
| Frontend | React (CRA) | 18.x |
| API Backend | FastAPI (async) | 0.115+ |
| ORM | SQLAlchemy (async) | 2.0 |
| Database | PostgreSQL | 15 |
| Cache / Queue | Redis | 7 |
| Blockchain | Ethereum (Hardhat) | 2.19+ |
| Smart Contract | Solidity | 0.8.20 |
| Web3 Library | Web3.py / ethers.js | 6.x / 6.x |
| File Storage | IPFS (Kubo) | 0.28+ |
| Containerization | Docker Compose | 2.x |

Table 1: Technology Stack

# 2 System Architecture

## 2.1 High-Level Architecture

## 2.2 Request Lifecycle

1. User interacts with React SPA in browser.
2. SPA sends HTTP REST request with JWT Bearer token to FastAPI backend.
3. Backend validates JWT, executes business logic, writes to PostgreSQL.

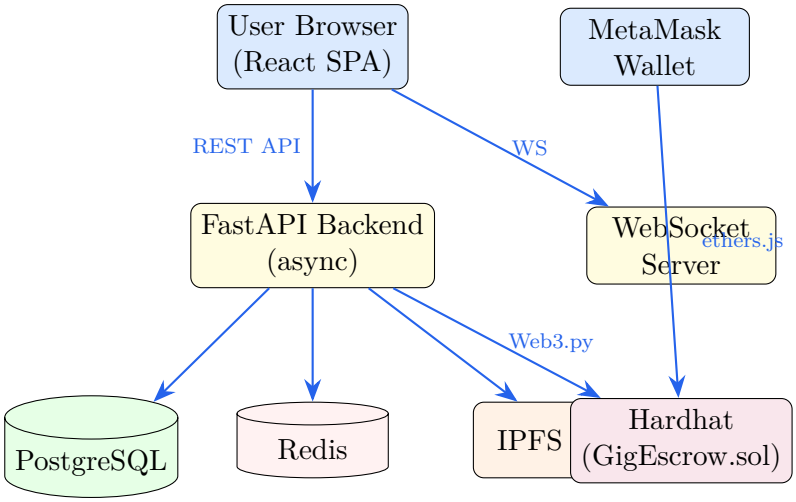


Figure 1: System Architecture Diagram

4. For blockchain operations, backend calls Hardhat via Web3.py using server-side private keys.
5. For user-signed transactions, MetaMask signs and submits directly to Hardhat.
6. Background event listener polls Hardhat every 5 seconds, syncing on-chain events to PostgreSQL.
7. Real-time updates delivered via WebSocket connections.

## 2.3 Docker Infrastructure

| Service | Image | Port | Purpose |
|---|---|---|---|
| PostgreSQL | postgres:15 | 5432 | Application database |
| Redis | redis:7 | 6379 | Caching, nonces, event state |
| Hardhat | node:20 | 8545 | Local Ethereum node |
| IPFS | ipfs/kubo:v0.28.0 | 5001, 8080 | Decentralized file storage |

Table 2: Docker Services

# 3 Database Design

## 3.1 Schema: `freeledger`

All tables reside in the `freeledger` PostgreSQL schema. The database uses SQLAlchemy 2.0 async ORM with `asyncpg` driver.

## 3.2 Entity Relationship Diagram

## 3.3 Core Tables

### 3.3.1 users

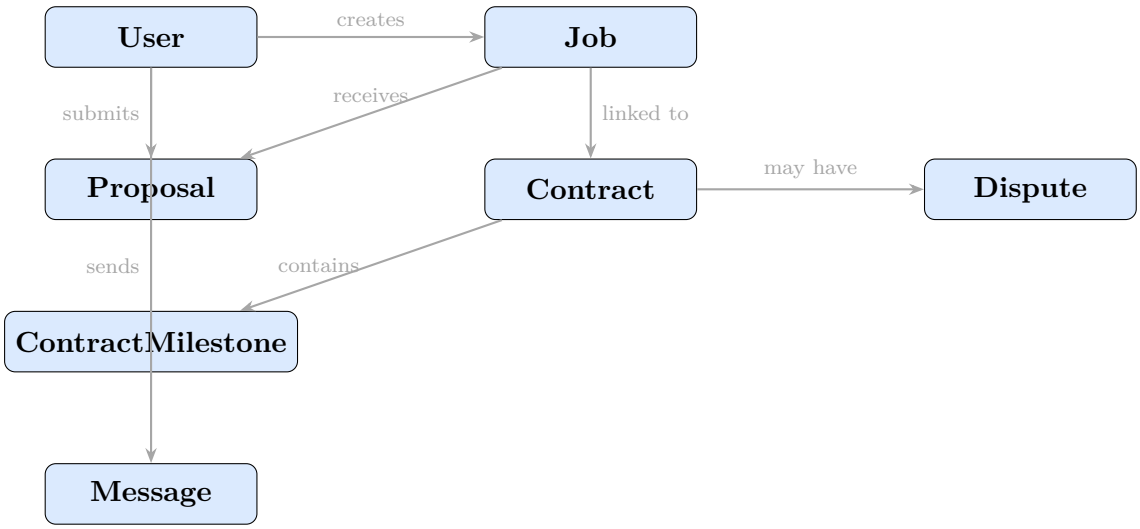


Figure 2: Simplified Entity Relationship Diagram

```
CREATE TABLE freeledger.users (
    id              VARCHAR(50) PRIMARY KEY,  -- usr_<hex>
    email           VARCHAR(255) UNIQUE,
    username        VARCHAR(100),
    password_hash   VARCHAR(255),
    auth_method     VARCHAR(20),    -- 'email' or 'wallet'
    wallet_address  VARCHAR(42),    -- MetaMask address (not unique)
    role            VARCHAR(20),    -- 'client', 'freelancer', 'admin'
    bio             TEXT,
    skills          JSONB,
    hourly_rate     DECIMAL(10,2),
    rating          DECIMAL(3,2) DEFAULT 0,
    avatar_cid      VARCHAR(100),
    totp_enabled    BOOLEAN DEFAULT FALSE,
    totp_secret     VARCHAR(64),
    totp_backup_codes JSONB,
    email_notifications BOOLEAN DEFAULT TRUE,
    location        VARCHAR(255),
    github_url      VARCHAR(500),
    linkedin_url    VARCHAR(500),
    portfolio_url   VARCHAR(500),
    is_active       BOOLEAN DEFAULT TRUE,
    created_at      TIMESTAMP DEFAULT NOW(),
    updated_at      TIMESTAMP DEFAULT NOW()
);
```

### 3.3.2 contracts

```
CREATE TABLE freeledger.contracts (
    id              VARCHAR(50) PRIMARY KEY,  -- ct_<hex>
    job_id          VARCHAR(50) REFERENCES jobs(id),
    client_id       VARCHAR(50) REFERENCES users(id),
    freelancer_id   VARCHAR(50) REFERENCES users(id),
    title           VARCHAR(255),
    description     TEXT,
    total_amount    DECIMAL(12,2),
    deadline        TIMESTAMP,
    terms_cid       VARCHAR(100),   -- IPFS CID for contract terms
    on_chain_id     INTEGER UNIQUE, -- GigEscrow contract ID
    contract_address VARCHAR(42),
    status          VARCHAR(30),    -- pending_signatures -> active -> completed
    client_signed   BOOLEAN DEFAULT FALSE,
    freelancer_signed BOOLEAN DEFAULT FALSE,
    created_at      TIMESTAMP DEFAULT NOW(),
    updated_at      TIMESTAMP DEFAULT NOW()
```

```
);
```

### 3.3.3 contract_milestones

```
CREATE TABLE freeledger.contract_milestones (
    id              VARCHAR(50) PRIMARY KEY,  -- ms_<hex>
    contract_id     VARCHAR(50) REFERENCES contracts(id),
    "index"         INTEGER,
    description     TEXT,
    amount          DECIMAL(12,2),
    due_date        TIMESTAMP,
    deliverable_cid VARCHAR(100),   -- IPFS CID of deliverable
    submission_notes TEXT,
    rejection_reason TEXT,
    status          VARCHAR(20),    -- pending -> submitted -> approved/rejected
    submitted_at    TIMESTAMP,
    approved_at     TIMESTAMP,
    created_at      TIMESTAMP DEFAULT NOW(),
    UNIQUE(contract_id, "index")
);
```

## 3.4 Enums

| Enum Type | Values |
|---|---|
| UserRole | client, freelancer, admin |
| AuthMethod | email, wallet |
| ContractStatus | draft, pending_review, pending_signatures, pending_funding, active, delivered, re |
| MilestoneStatus | pending, submitted, approved, rejected, paid |
| DisputeStatus | open, under_review, resolved |
| DisputeDecision | refund, release |

Table 3: Database Enum Types

# 4 Smart Contract Design

## 4.1 Overview

The `GigEscrow` contract is a Solidity 0.8.20 smart contract deployed on the Hardhat local network. It implements a milestone-based escrow system with a 2.5% platform fee, using OpenZeppelin's `ReentrancyGuard` and `Ownable` for security.

## 4.2 State Variables

```
uint256 public constant PLATFORM_FEE_BPS = 250;  // 2.5%
uint256 public contractCounter;

mapping(uint256 => EscrowContract) public contracts;
mapping(uint256 => mapping(uint256 => Milestone)) public milestones;
mapping(address => uint256[]) public clientContracts;
mapping(address => uint256[]) public freelancerContracts;
mapping(uint256 => bool) public contractExists;
```

## 4.3 Data Structures

```
enum ContractStatus { Created, InProgress, Completed, Cancelled, Disputed }
enum MilestoneStatus { Pending, Funded, Submitted, Approved, Rejected }

struct EscrowContract {
    address client;
    address freelancer;
    string title;
    string termsCID;
    uint256 totalAmount;
    uint256 deadline;
    ContractStatus status;
    uint256 milestoneCount;
    uint256 completedMilestones;
}

struct Milestone {
    string description;
    uint256 amount;
    string deliverableCID;
    MilestoneStatus status;
    uint256 submittedAt;
    uint256 approvedAt;
}
```

## 4.4 Access Control

Four modifiers control function access:

```
modifier onlyClient(uint256 _contractId) {
    require(
        msg.sender == contracts[_contractId].client ||
        msg.sender == owner(),
        "Only client"
    );
    _;
}

modifier onlyFreelancer(uint256 _contractId) {
    require(
        msg.sender == contracts[_contractId].freelancer ||
        msg.sender == owner(),
        "Only freelancer"
    );
    _;
}

modifier onlyContractParty(uint256 _contractId) {
    require(
        msg.sender == contracts[_contractId].client ||
```

```
        msg.sender == contracts[_contractId].freelancer ||
        msg.sender == owner(),
        "Not a contract party or owner"
    );
    _;
}

modifier contractExistsMod(uint256 _contractId) {
    require(contractExists[_contractId], "Contract does not exist");
    _;
}
```

**Design Decision:** Both `onlyClient` and `onlyFreelancer` allow the `owner()` (the platform backend) to execute operations. This enables server-side transaction signing for operations like milestone submission and dispute resolution without requiring MetaMask for every action.

## 4.5 Contract Lifecycle Functions

### 4.5.1 createContract

Deploys a new escrow contract with milestones. Validates that milestone amounts sum to the total.

```
function createContract(
    address _client,
    address _freelancer,
    string memory _title,
    string memory _termsCID,
    uint256 _totalAmount,
    uint256 _deadline,
    string[] memory _milestoneDescriptions,
    uint256[] memory _milestoneAmounts
) external returns (uint256) {
    require(_client != address(0));
    require(_client != _freelancer);
    require(_milestoneDescriptions.length > 0);
    require(totalMilestoneAmount == _totalAmount);

    contractCounter++;
    // Store EscrowContract + Milestone structs
    // Emit ContractCreated + MilestoneAdded events
    return contractId;
}
```

### 4.5.2 fundContract

Client funds the contract with ETH. Transitions status from `Created` to `InProgress` and all milestones from `Pending` to `Funded`.

```
function fundContract(uint256 _contractId)
    external payable onlyClient(_contractId)
```

```
{
    require(escrow.status == ContractStatus.Created);
    require(msg.value == escrow.totalAmount);

    escrow.status = ContractStatus.InProgress;
    for (uint256 i = 0; i < escrow.milestoneCount; i++) {
        milestones[_contractId][i].status = MilestoneStatus.Funded;
    }
}
```

### 4.5.3 submitMilestone

Freelancer submits deliverable IPFS CID. Requires `InProgress` status and `Funded` milestone.

```
function submitMilestone(
    uint256 _contractId,
    uint256 _milestoneIndex,
    string memory _deliverableCID
) external onlyFreelancer(_contractId) {
    require(escrow.status == ContractStatus.InProgress);
    require(milestone.status == MilestoneStatus.Funded);
    require(bytes(_deliverableCID).length > 0);

    milestone.deliverableCID = _deliverableCID;
    milestone.status = MilestoneStatus.Submitted;
    milestone.submittedAt = block.timestamp;
}
```

### 4.5.4 approveMilestone

Client approves a submitted milestone. Transfers payment to freelancer minus 2.5% platform fee. Auto-completes contract when all milestones are approved.

```
function approveMilestone(
    uint256 _contractId,
    uint256 _milestoneIndex
) external onlyClient(_contractId) nonReentrant {
    require(milestone.status == MilestoneStatus.Submitted);

    milestone.status = MilestoneStatus.Approved;
    escrow.completedMilestones++;

    uint256 fee = (milestone.amount * PLATFORM_FEE_BPS) / 10000;
    uint256 payout = milestone.amount - fee;

    payable(escrow.freelancer).transfer(payout);
    payable(owner()).transfer(fee);

    if (escrow.completedMilestones == escrow.milestoneCount) {
        _completeContract(_contractId);
```

```
    }
}
```

#### 4.5.5 resolveDispute

Admin-only function to resolve disputes. Releases funds to freelancer or refunds to client.

```
function resolveDispute(
    uint256 _contractId,
    bool _releaseToFreelancer
) external onlyOwner nonReentrant {
    require(escrow.status == ContractStatus.Disputed);

    if (_releaseToFreelancer) {
        uint256 balance = address(this).balance;
        uint256 fee = (balance * PLATFORM_FEE_BPS) / 10000;
        payable(escrow.freelancer).transfer(balance - fee);
        payable(owner()).transfer(fee);
    } else {
        payable(escrow.client).transfer(address(this).balance);
    }
}
```

### 4.6 Events

| Event | Parameters |
|---|---|
| ContractCreated | contractId (indexed), client (indexed), freelancer (indexed), totalAmount |
| MilestoneAdded | contractId (indexed), milestoneIndex |
| MilestoneSubmitted | contractId (indexed), milestoneIndex, deliverableCID |
| MilestoneApproved | contractId (indexed), milestoneIndex, amount |
| MilestoneRejected | contractId (indexed), milestoneIndex |
| ContractCompleted | contractId (indexed) |
| ContractCancelled | contractId (indexed) |
| DisputeRaised | contractId (indexed), raisedBy |
| DisputeResolved | contractId (indexed), winner |
| FreelancerSet | contractId (indexed), freelancer (indexed) |

Table 4: Smart Contract Events

### 4.7 State Machine

## 5 Authentication System

FreeLedger supports three authentication methods: email/password, MetaMask wallet (SIWE), and optional TOTP two-factor authentication.

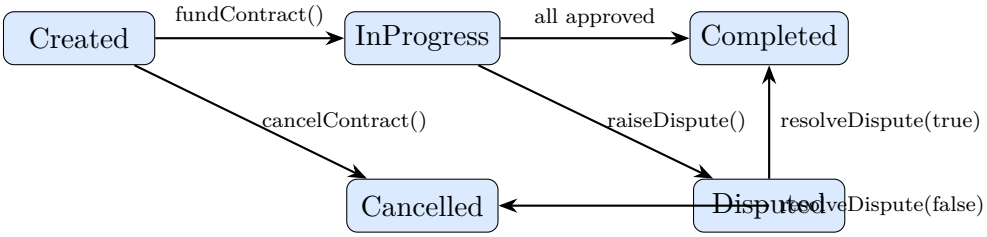


Figure 3: Contract State Machine

## 5.1 Email/Password Authentication

### 5.1.1 Registration

```python
@router.post("/register", response_model=TokenResponse)
async def register(request: RegisterRequest, db: AsyncSession =
    Depends(get_db)):
    user = User(
        email=request.email.lower(),
        password_hash=hash_password(request.password),  # bcrypt
        auth_method="email",
        username=request.username or request.email.split("@")[0],
        role=request.role.lower(),
    )
    db.add(user)
    await db.flush()
    access_token = create_access_token(user.id)    # JWT, 30-min expiry
    refresh_token = create_refresh_token(user.id)  # JWT, 7-day expiry
    return TokenResponse(access_token=access_token,
    refresh_token=refresh_token, ...)
```

### 5.1.2 Login with TOTP Flow

When TOTP 2FA is enabled, the login endpoint returns a temporary token instead of a full JWT:

```python
@router.post("/login", response_model=TokenResponse)
async def login(request: LoginRequest, db: AsyncSession = Depends(get_db)):
    user = # SELECT by email
    if not verify_password(request.password, user.password_hash):
        raise HTTPException(status_code=401)

    if user.totp_enabled and user.totp_secret:
        totp_token = create_temp_token(user.id)  # 5-min JWT,
    type="totp_pending"
        return TokenResponse(requires_totp=True, totp_token=totp_token)

    return TokenResponse(access_token=..., refresh_token=..., user=...)
```

## 5.2 MetaMask SIWE Authentication

Sign-In with Ethereum (SIWE) flow using ECDSA signature verification:

```
@router.post("/auth/wallet/login")
async def wallet_login(request, body, db):
    # 1. Retrieve single-use nonce from Redis
    nonce = await redis.get(f"nonce:{address}")
    await redis.delete(f"nonce:{address}")

    # 2. Verify ECDSA signature
    message = encode_defunct(text=nonce)
    recovered = Account.recover_message(message, signature=signature)
    if recovered.lower() != address:
        raise HTTPException(status_code=401)

    # 3. Three flows:
    #    A: Pure MetaMask (no email) -- auto-generate credentials
    #    B: Wallet + email signup -- create new user
    #    C: Link wallet to existing email -- verify password first

    return { access_token, refresh_token, user }
```

**Wallet Uniqueness Rule:** A single wallet address can connect to a maximum of 2 accounts (1 freelancer + 1 client). The `wallet_address` column is *not* unique; the uniqueness check is enforced at the application layer.

## 5.3 TOTP Two-Factor Authentication

```
# Rate limiting: 5 attempts per 60 seconds per user
totp_rate_limit: dict[str, list[float]] = defaultdict(list)

@router.post("/auth/totp/validate")
async def totp_validate(data: TOTPValidateRequest, db=Depends(get_db)):
    user_id = verify_temp_token(data.totp_token)

    # Rate limit check
    rate_key = f"totp_{user_id}"
    totp_rate_limit[rate_key] = [t for t in totp_rate_limit[rate_key]
                                 if now - t < TOTP_RATE_WINDOW]
    if len(totp_rate_limit[rate_key]) >= TOTP_MAX_ATTEMPTS:
        raise HTTPException(status_code=429)

    if not verify_code(user.totp_secret, data.code):
        totp_rate_limit[rate_key].append(now)
        raise HTTPException(status_code=401)

    # Issue full JWT
    access_token = create_access_token(user_id, backup_login=used_backup_code)
    return TokenResponse(access_token=access_token, ...)
```

**Backup Codes:** 8 single-use codes in `XXXX-XXXX` format, SHA-256 hashed. A `backup_login` flag is encoded in the JWT to allow the user to reset 2FA via the `/auth/totp/reset` endpoint.

### 5.4 JWT Token Structure

```
# Access Token (30-minute expiry)
{
    "sub": "usr_a1b2c3d4",
    "exp": 1700000000,
    "type": "access",
    "iat": 1699998200,
    "jti": "unique_token_id",
    "backup_login": false  # true when logged in via backup code
}

# Refresh Token (7-day expiry)
{
    "sub": "usr_a1b2c3d4",
    "exp": 1700604800,
    "type": "refresh",
    "iat": 1699998200
}

# TOTP Temp Token (5-minute expiry)
{
    "sub": "usr_a1b2c3d4",
    "exp": 1700000300,
    "type": "totp_pending"
}
```

## 6 Core Business Flows

### 6.1 Job Posting and Discovery

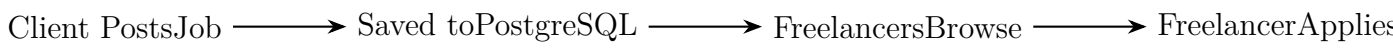


Figure 4: Job Posting Flow

```
@router.post("/jobs", response_model=JobResponse, status_code=201)
async def create_job(job_data: JobCreate, db=Depends(get_db),
                     current_user: User = Depends(get_current_user)):
    if current_user.role.value != "client":
        raise HTTPException(status_code=403)

    job = Job(
        client_id=current_user.id,
```

```
        title=job_data.title,
        description=job_data.description,
        budget=job_data.budget,
        category=job_data.category,
        skills=job_data.skills,
        duration_days=job_data.duration_days,
    )
    db.add(job)
    await db.commit()
    return job
```

Jobs support filtering by status, category, search term, and client ID. The listing endpoint computes `applicants_count` and `has_ired` via SQL JOINs.

## 6.2 Proposal Submission and Acceptance

### 6.2.1 Freelancer Submits Proposal

```
@router.post("/proposals", response_model=ProposalResponse, status_code=201)
async def create_proposal(data: ProposalCreate, db, current_user):
    if current_user.role.value != "freelancer":
        raise HTTPException(status_code=403)

    # Enforce one proposal per freelancer per job
    existing = await db.execute(
        select(Proposal).where(
            Proposal.job_id == data.job_id,
            Proposal.freelancer_id == current_user.id,
        )
    )
    if existing.scalar_one_or_none():
        raise HTTPException(status_code=400, detail="Already applied")

    proposal = Proposal(
        job_id=data.job_id,
        freelancer_id=current_user.id,
        cover_letter=data.cover_letter,
        bid_amount=data.bid_amount,
        estimated_days=data.estimated_days,
    )
    db.add(proposal)
    await db.commit()
    return proposal
```

### 6.2.2 Client Accepts Proposal

This is the most complex flow in the system. It orchestrates database updates, on-chain deployment, IPFS upload, and real-time messaging:

```
@router.post("/proposals/{proposal_id}/accept",
    response_model=ProposalResponse)
```

```python
async def accept_proposal(proposal_id, db, current_user):
    # 1. Find contract for this job without a freelancer
    contract = # SELECT Contract WHERE job_id AND freelancer_id IS NULL

    # 2. Assign freelancer to contract
    contract.freelancer_id = proposal.freelancer_id
    proposal.status = "accepted"

    # 3. Reject all other pending proposals
    for p in remaining_proposals:
        p.status = "rejected"

    # 4. Deploy or update on-chain contract
    if contract.on_chain_id is not None:
        set_freelancer_on_chain(contract.on_chain_id, freelancer_wallet)
    else:
        # Upload terms to IPFS
        ipfs_result = await upload_contract_terms(terms_doc)
        # Deploy new escrow contract via backend
        on_chain = await asyncio.to_thread(create_contract_on_chain, ...)
        contract.on_chain_id = on_chain["on_chain_id"]

    # 5. Send hire notification via WebSocket
    hire_msg = Message(content=f"You've been hired for \"{job.title}\"!")
    await msg_manager.broadcast_to_both(client_id, freelancer_id, {...})

    return proposal
```

## 6.3 Contract Signing and Funding

### 6.3.1 Signing

Both parties must sign the contract before funding:

```python
async def sign_contract(db, contract_id, user_id):
    if contract.client_id == user_id:
        contract.client_signed = True
    elif contract.freelancer_id == user_id:
        contract.freelancer_signed = True

    if contract.client_signed and contract.freelancer_signed:
        if contract.on_chain_id is not None:
            contract.status = ContractStatus.pending_funding
```

### 6.3.2 Funding

The client funds the contract, which transitions it to `active`:

```python
async def fund_contract(db, contract_id, user_id):
    on_chain_state = await asyncio.to_thread(get_contract_state, on_chain_id)
    if on_chain_status == 1:  # InProgress on-chain
```

```
        contract.status = ContractStatus.active
```

## 6.4 Milestone Lifecycle

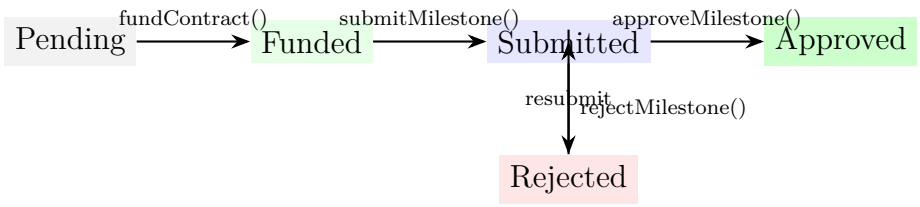


Figure 5: Milestone State Machine

### 6.4.1 Freelancer Submits Milestone

```python
@router.post("/{contract_id}/milestones/{index}/submit")
async def submit_milestone(contract_id, index, data, db, current_user):
    ms.status = MilestoneStatus.submitted
    ms.deliverable_cid = data.deliverable_cid
    ms.submitted_at = func.now()

    # Submit on-chain (non-fatal if blockchain is down)
    if contract.on_chain_id is not None:
        try:
            await asyncio.to_thread(
                blockchain_service.submit_milestone_on_chain, ...)
        except Exception as exc:
            print(f"[WARN] On-chain submission failed: {exc}")

    # Notify client
    await create_notification(db, user_id=contract.client_id,
        type="milestone_submitted", ...)
```

### 6.4.2 Client Approves Milestone (Payment Release)

```python
@router.post("/{contract_id}/milestones/{index}/approve")
async def approve_milestone(contract_id, index, db, current_user):
    ms.status = MilestoneStatus.approved
    ms.approved_at = func.now()

    # Check if all milestones done -> complete contract
    all_ms = await db.execute(select(ContractMilestone)...)
    all_done = all(m.status == MilestoneStatus.approved for m in all_ms)
    if all_done:
        contract.status = ContractStatus.completed

    await create_notification(db, user_id=contract.freelancer_id,
```

```
        type="milestone_approved",
        message="Payment released!", ...)
```

**On-Chain Payment:** The actual ETH transfer happens via the smart contract's `approveMilestone()` function. The backend updates the database state; the frontend triggers the MetaMask transaction for payment release.

## 6.5 Payment and Escrow Flow

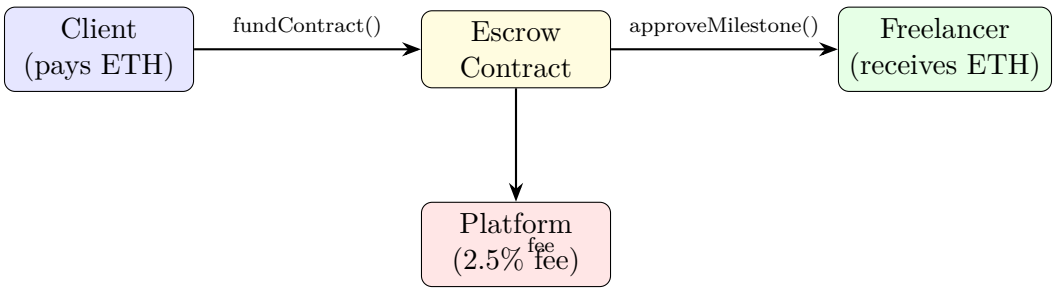


Figure 6: Payment Flow

**Fee Structure:**

- Platform fee: 2.5% (250 basis points)
- Freelancer receives: `amount - (amount * 250 / 10000)`
- Platform receives: `amount * 250 / 10000`

## 6.6 Dispute Resolution

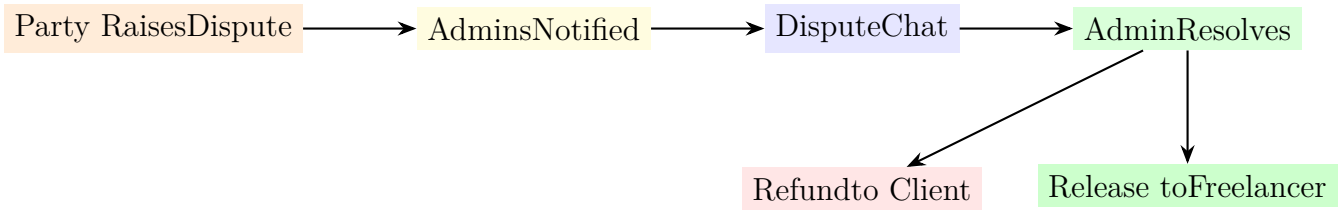


Figure 7: Dispute Resolution Flow

```python
# Raising a dispute
@router.post("/contracts/{contract_id}/disputes")
async def create_dispute(contract_id, data, db, current_user):
    # Validate contract state
    if contract.status in [ContractStatus.completed,
    ContractStatus.cancelled]:
        raise HTTPException(status_code=400)

    # Raise on-chain
    if contract.on_chain_id is not None:
        await asyncio.to_thread(blockchain_service.raise_dispute_on_chain,
    ...)

```

```
    dispute = Dispute(contract_id=contract_id, raised_by=current_user.id,
                      status=DisputeStatus.open, reason=data.reason)
    contract.status = ContractStatus.disputed

    # Notify all admins
    admins = await db.execute(select(AdminAccount))
    for admin in admins.scalars().all():
        await create_notification(db, user_id=admin.user_id,
            type="dispute_raised", ...)

# Resolving a dispute (admin only)
@router.post("/disputes/{dispute_id}/resolve")
async def resolve_dispute(dispute_id, data, db, current_user):
    await _require_admin(current_user, db)

    # Resolve on-chain
    await asyncio.to_thread(blockchain_service.resolve_dispute_on_chain,
        contract.on_chain_id, data.release_to_freelancer)

    dispute.status = DisputeStatus.resolved
    dispute.decision = "release" if data.release_to_freelancer else "refund"
    contract.status = (ContractStatus.completed
                       if data.release_to_freelancer
                       else ContractStatus.cancelled)

    # Notify both parties
    await create_notification(db, user_id=contract.client_id, ...)
    await create_notification(db, user_id=contract.freelancer_id, ...)
```

# 7 API Reference

## 7.1 Authentication Endpoints

| Method<br>Description | Endpoint | Auth |
|---|---|---|
| POST<br>Register with email/password | /api/auth/register | None |
| POST<br>Login, returns TOTP token if 2FA enabled | /api/auth/login | None |
| POST<br>Admin login (username-based) | /api/auth/admin/login | None |
| POST<br>Refresh JWT pair | /api/auth/refresh | Refresh Token |
| GET<br>Get current user profile | /api/auth/me | Bearer |
| POST<br>Generate SIWE nonce | /api/auth/wallet/challenge | None |

| Method<br>Description | Endpoint | Auth |
|---|---|---|
| GET<br>Check wallet account status | /api/auth/wallet/status | None |
| POST<br>Verify SIWE signature, login | /api/auth/wallet/login | None |
| GET<br>Check 2FA status | /api/auth/totp/status | Bearer |
| POST<br>Generate TOTP secret + QR + backup codes | /api/auth/totp/setup | Bearer |
| POST<br>Verify code to enable 2FA | /api/auth/totp/verify | Bearer |
| POST<br>Validate TOTP during login (rate-limited) | /api/auth/totp/validate | None |
| POST<br>Disable 2FA | /api/auth/totp/disable | Bearer |
| POST<br>Reset 2FA (requires backup_login JWT) | /api/auth/totp/reset | Bearer* |

## 7.2 Job Endpoints

| Method<br>Description | Endpoint | Auth |
|---|---|---|
| GET<br>List jobs with filters and pagination | /api/jobs | Bearer |
| POST<br>Create a new job | /api/jobs | Bearer (client) |
| GET<br>Get job detail with milestones | /api/jobs/{id} | Bearer |
| PUT<br>Update job | /api/jobs/{id} | Bearer (owner) |
| DELETE<br>Delete job | /api/jobs/{id} | Bearer (owner/admin) |
| PUT<br>Set on-chain contract ID | /api/jobs/{id}/on-chain-id | Bearer (owner) |

## 7.3 Contract Endpoints

| Method<br>Description | Endpoint | Auth |
|---|---|---|
| POST<br>Create contract, deploy on-chain | /api/contracts | Bearer (client) |
| GET | /api/contracts | Bearer |

| Method<br>Description | Endpoint | Auth |
|---|---|---|
| List user's contracts with milestones<br>GET | /api/contracts/{id} | Bearer (party) |
| Get contract detail<br>POST | /api/contracts/{id}/sign | Bearer (party) |
| Sign contract<br>POST | /api/contracts/{id}/fund | Bearer (client) |
| Fund contract<br>POST | /api/contracts/{id}/milestones/{i}/submit | Bearer (freelanc |
| Submit milestone<br>POST | /api/contracts/{id}/milestones/{i}/approve | Bearer (client) |
| Approve milestone<br>POST | /api/contracts/{id}/milestones/{i}/reject | Bearer (client) |
| Reject milestone<br>POST | /api/contracts/{id}/disputes | Bearer (party) |
| Raise dispute | | |

## 7.4 Messaging Endpoints

| Method<br>Description | Endpoint | Auth |
|---|---|---|
| WS<br>WebSocket for real-time messaging | /api/messages/ws/{user_id} | None |
| POST<br>Send message to another user | /api/messages/send | Bearer |
| GET<br>Get all messages for current user | /api/messages/inbox | Bearer |
| POST<br>Mark single message as read | /api/messages/{id}/read | Bearer |
| POST<br>Mark entire thread as read | /api/messages/thread/{partner_id}/read | Bearer |
| GET<br>Get total unread count | /api/messages/unread-count | Bearer |
| DELETE<br>Delete thread (guarded by contract status) | /api/messages/thread/{partner_id} | Bearer |

## 7.5 Admin Endpoints

| Method<br>Description | Endpoint | Auth |
|---|---|---|
| GET<br>Platform statistics | /api/admin/dashboard | Admin |

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| GET | /api/admin/reports | Admin | Comprehensive analytics |
| GET | /api/admin/users | Admin | List non-admin users |
| POST | /api/admin/users | Admin | Create user |
| PUT | /api/admin/users/{id} | Admin | Update user |
| DELETE | /api/admin/users/{id} | Admin | Delete user |
| GET | /api/admin/jobs | Admin | List all jobs |
| GET | /api/admin/contracts | Admin | List all contracts |
| GET | /api/admin/disputes | Admin | List disputes with contract details |
| GET | /api/admin/audit-logs | Admin | Query audit logs |
| GET | /api/disputes | Bearer | List disputes for user's contracts |
| POST | /api/disputes/{id}/resolve | Admin | Resolve dispute |

# 8 Backend Services

## 8.1 Blockchain Service

Wraps all Web3.py interactions with the GigEscrow contract. Uses lazy singletons for `Web3` and contract instances.

```
# Key functions
create_contract_on_chain(...)      # Deploy new escrow
set_freelancer_on_chain(...)       # Assign freelancer address
fund_contract_on_chain(...)        # Fund with ETH
submit_milestone_on_chain(...)     # Submit deliverable CID
approve_milestone_on_chain(...)    # Release payment
reject_milestone_on_chain(...)     # Reset milestone
raise_dispute_on_chain(...)        # Set status to Disputed
resolve_dispute_on_chain(...)      # Release or refund
get_contract_state(on_chain_id)    # Read on-chain details
build_contract_tx(fn, address)     # Auto-estimate gas (1.5x buffer)
sign_and_send(tx, private_key)     # Sign and broadcast
```

**Resilience Design:** On-chain failures (submit, reject) are non-fatal. The database always updates regardless of blockchain state. This ensures the application remains functional when the Hardhat node restarts.

## 8.2 Event Listener

Background service polling Hardhat every 5 seconds:

```python
async def poll_events():
    while True:
        last_block = await _get_last_block(redis)
        current_block = w3.eth.block_number

        if current_block <= last_block:
            await asyncio.sleep(POLL_INTERVAL)
            continue

        # Poll 8 event types
        event_handlers = [
            "ContractCreated", "MilestoneSubmitted",
            "MilestoneApproved", "MilestoneRejected",
            "ContractCompleted", "ContractCancelled",
            "DisputeRaised", "DisputeResolved",
        ]

        # Sync on-chain events to PostgreSQL
        for event_name, events in all_events:
            for evt in events:
                # Route to handler, update DB
                pass

        await _set_last_block(redis, current_block)
```

## 8.3 IPFS Service

```python
upload_file(file, filename)          # Upload to IPFS via /api/v0/add
upload_file_bytes(data, filename)    # Upload bytes via temp file
download_file(cid)                   # Download from IPFS via /api/v0/cat
pin_file(cid)                        # Pin content
file_exists(cid)                     # Check existence
upload_contract_terms(data)          # Serialize + upload contract terms JSON
```

## 8.4 Rate Limiting Middleware

Redis sliding window rate limiter with per-IP tracking:

| Endpoint Category | Limit |
|---|---|
| Authentication (login, register) | 10 requests/minute |
| Write operations (POST, PUT, DELETE) | 60 requests/minute |
| Read operations (GET) | 300 requests/minute |
| Health checks, notifications | Excluded |

Table 10: Rate Limits

Returns `429 Too Many Requests` with `Retry-After`, `X-RateLimit-Limit`, and `X-RateLimit-Rema` headers.

### 8.5 Caching Middleware

Backend adds `Cache-Control` headers:

| Resource | Cache Duration |
|---|---|
| Jobs, Users, Reports | 30s (stale-while-revalidate) |
| Contracts | 10s |
| /users/me | No-cache |
| Auth, Messages, Notifications | No-store |

Table 11: Cache Headers

## 9 Frontend Architecture

### 9.1 Application Structure

```
frontend/src/
  App.js                    # Route definitions (client, freelancer, admin)
  context/AppContext.js     # Global state (wallet, user, auth)
  hooks/useAuth.js          # MetaMask authentication hook
  services/
    api.js                  # Axios with JWT interceptor, 15s request cache
    auth.js                 # MetaMask SIWE helpers
    web3.js                 # ethers.js v6 providers
    ipfs.js                 # IPFS upload/gateway
    contractAbi.js          # GigEscrow ABI constant
  pages/
    Login.js                # Combined Sign In/Create Account + TOTP flow
    Landing.js              # Marketing landing page
    client/                 # Client-specific pages
    freelancer/             # Freelancer-specific pages
    admin/                  # Admin-specific pages
    shared/Messages.js      # Shared messaging component
  components/
    shared/                 # TOTPSettings, NotificationBell, Skeleton
    client/Navbar.js        # Client navigation
    freelancer/Navbar.js    # Freelancer navigation
    admin/Navbar.js         # Admin navigation
```

### 9.2 Route Map

### 9.3 API Client with Caching

```
# Frontend API client (api.js)
import axios from 'axios';

const cache = new Map();
const CACHE_TTL = 15000; // 15 seconds

const api = axios.create({ baseURL: config.apiUrl });

```

| Path | Component |
|---|---|
| *Client Routes* | |
| /client/dashboard | ClientDashboard |
| /client/explore-jobs | ExploreJobs |
| /client/browse-freelancers | BrowseFreelancers |
| /client/my-contracts | MyContracts |
| /client/profile | ClientProfile |
| /client/messages | Messages |
| *Freelancer Routes* | |
| /freelancer/dashboard | FreelancerDashboard |
| /freelancer/jobs | FindJobs |
| /freelancer/contracts | MyContracts |
| /freelancer/my-profile | MyProfile |
| /freelancer/messages | Messages |
| *Admin Routes (port 3001)* | |
| /dashboard | AdminDashboard |
| /users | AdminUsers |
| /user-search | UserSearch |
| /jobs | AdminJobs |
| /proposals | AdminProposals |
| /contracts | AdminContracts |
| /disputes | AdminDisputes |
| /audit-logs | AuditLogs |
| /reports | Report |
| /profile | AdminProfile |

Table 12: Frontend Routes

```
// Custom adapter: short-circuit HTTP on cache hit
api.defaults.adapter = async (config) => {
    if (config.method === 'get') {
        const key = config.url + JSON.stringify(config.params || {});
        const cached = cache.get(key);
        if (cached && Date.now() - cached.time < CACHE_TTL) {
            return { data: cached.data, status: 200, statusText: 'OK',
                     headers: {}, config };
        }
    }
    // ... normal axios request
    // Auto-invalidate related entries on mutations
};

// Clear on logout
api.clearCache = () => cache.clear();
```

## 9.4 WebSocket Integration

Real-time messaging uses WebSocket connections:

```
// Backend ConnectionManager
class ConnectionManager:
    def __init__(self):
        self.active: dict[str, WebSocket] = {}

    async def connect(self, websocket, user_id):
        await websocket.accept()
        self.active[user_id] = websocket

    async def broadcast_to_both(self, user_a, user_b, message):
        for uid in [user_a, user_b]:
            if uid in self.active:
                await self.active[uid].send_json(message)
```

Events: `new_message`, `message_read`, `thread_deleted`, `dispute_message`, `dispute_chat_deleted`

# 10 Security

## 10.1 Authentication Security

- Passwords hashed with bcrypt (cost factor from passlib defaults)
- JWT tokens signed with HS256, 30-minute access / 7-day refresh expiry
- SIWE nonces stored in Redis with 5-minute TTL, single-use
- TOTP 2FA with rate limiting (5 attempts/60 seconds)
- Backup codes SHA-256 hashed, single-use
- JWT `backup_login` flag prevents 2FA bypass after recovery

## 10.2 Smart Contract Security

- OpenZeppelin `ReentrancyGuard` on payment functions (`approveMilestone`, `resolveDispute`, `cancelContract`)
- `Ownable` for admin-only operations
- Input validation: zero-address checks, self-contract prevention, array length matching
- State machine enforcement: each function validates current contract status
- Exact funding amount validation: `msg.value == escrow.totalAmount`

## 10.3 API Security

- CORS configuration with explicit origin whitelist
- Redis sliding window rate limiting per IP
- Role-based access control (RBAC) on all endpoints
- Contract party verification: users can only access their own contracts
- Chat deletion guarded by contract/dispute status

## 10.4 Infrastructure Security

- Database credentials in environment variables (not committed)
- Private keys stored in `.env`, never exposed to frontend

- IPFS content pinned to prevent garbage collection
- Docker containers with memory limits

# 11 Deployment

## 11.1 Quick Start

```
# Clone repository
git clone https://github.com/Dumb-o/CapstoneV3.git
cd CapstoneV3

# Start all services (auto-installs Python/Node.js if missing)
chmod +x start.sh
./start.sh
```

The `start.sh` script:

1. Checks/installs Python 3 and Node.js (via brew/apt/yum/winget/choco)
2. Starts Docker containers (PostgreSQL, Redis, Hardhat, IPFS)
3. Creates Python venv and installs dependencies
4. Runs startup migrations (schema evolution)
5. Starts FastAPI backend on port 8000
6. Installs frontend dependencies
7. Starts React dev server on port 3000

## 11.2 Environment Configuration

```
# backend/.env
DATABASE_URL=postgresql+asyncpg://freeledger:freeledger_dev@localhost:5432/freeledger
REDIS_URL=redis://localhost:6379
RPC_URL=http://127.0.0.1:8545
CONTRACT_ADDRESS=0xDc64a140Aa3E981100a9becA4E685f962f0cF6C9
CLIENT_PRIVATE_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
IPFS_API_URL=http://127.0.0.1:5001

# frontend/.env
REACT_APP_CONTRACT_ADDRESS=0xDc64a140Aa3E981100a9becA4E685f962f0cF6C9
REACT_APP_BLOCKCHAIN_RPC=http://127.0.0.1:8545
REACT_APP_CHAIN_ID=31337
REACT_APP_API_URL=http://127.0.0.1:8000/api
REACT_APP_IPFS_GATEWAY=http://localhost:8080
```

## 11.3 Startup Migrations

The backend runs inline database migrations at startup, tracked via a `schema_migrations` table:

# Appendix

## 11.4 Project Directory Structure

| Version | Description |
|---|---|
| v2 | Clear stale on_chain_ids after contract redeployment |
| v3 | Relax wallet_address UNIQUE constraint (max 2 accounts per wallet) |
| v4 | Clear on_chain_ids after Hardhat restart |
| v5 | Add email_notifications column to users |

Table 13: Database Migrations

```
CapstoneV3/
  start.sh                    # One-command startup script
  admin.sh                    # Admin account manager
  fund.sh                     # Test wallet funder
  README.md                   # Project documentation
  backend/
    app/
      main.py                 # FastAPI application entry
      config.py               # Pydantic settings
      database.py             # SQLAlchemy async engine
      models.py               # ORM models (11 tables)
      schemas.py              # Pydantic request/response schemas
      redis_client.py         # Redis async client
      routers/                # 14 API router modules
      services/               # 10 service modules
      middleware/              # Rate limiting + caching
  frontend/
    src/
      App.js                  # Route definitions
      context/AppContext.js   # Global state provider
      hooks/useAuth.js        # MetaMask auth hook
      services/               # API client, web3, IPFS
      pages/                  # Page components
      components/             # Shared UI components
      css/                    # Scoped stylesheets
  contracts/
    contracts/GigEscrow.sol   # Solidity smart contract
    scripts/deploy.js         # Deployment script
  docker/
    docker-compose.yml        # Infrastructure containers
  scripts/
    fund-wallet.js            # Test ETH funder
```